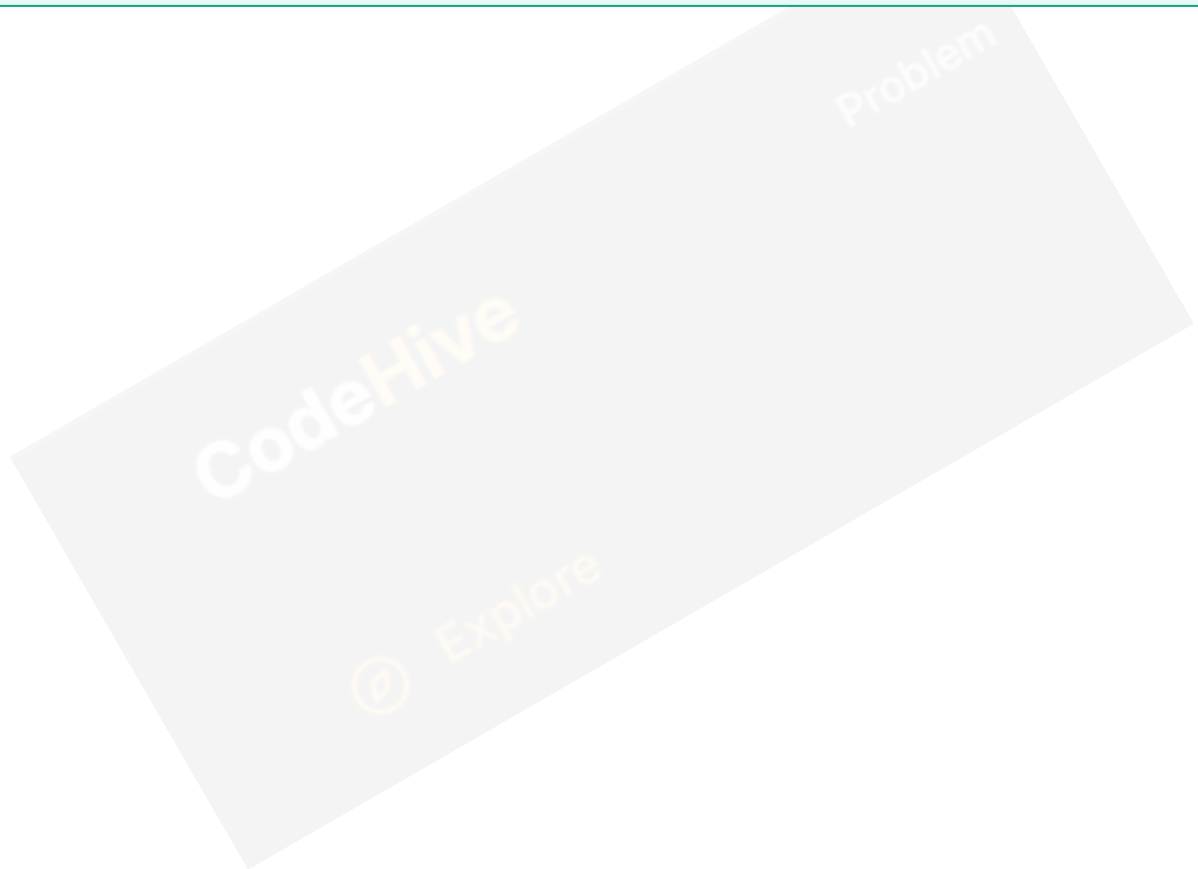


typedef in C

The typedef keyword is used to create an **alias** or a custom name for an existing data type. It is a powerful tool to make C code more intuitive and readable.

CodeHive Memo: Think of it as a nickname. A person named "Robert" might be called "Bob". Both represent the same person, just as `int` and `count` can represent the same data type.



1. Basic Syntax

The syntax for typedef is straightforward:

```
typedef existing_datatype new_name;
```

Example:

```
typedef int number;  
number a = 10;  
number b = 20;
```

Here, number is not a new type; it is just another name for int.

CodeHive

@ Explore

2. typedef with Structures

By default, in C, you must use the `struct` keyword every time you declare a variable. `typedef` removes this redundancy.

```
typedef struct {  
    int roll;  
    float marks;  
} Student;
```

Now, you can simply write: `Student s1, s2;`

CodeHive

@ Explore

3. Before vs. After typedef

Without typedef:

```
struct Student s1;  
struct Student s2;
```

With typedef:

```
Student s1;  
Student s2;
```

This drastically cleans up your code, especially in large projects with hundreds of structure variables.

CodeHive

@ Explore

4. typedef with Pointers

Pointer syntax can sometimes be confusing. typedef simplifies pointer declarations.

```
typedef int* IntPtr;  
IntPtr p1, p2;
```

Without typedef, if you write `int* p1, p2;`, `p1` is a pointer but `p2` is just an integer. Using `IntPtr` makes both `p1` and `p2` pointers correctly.

CodeHive

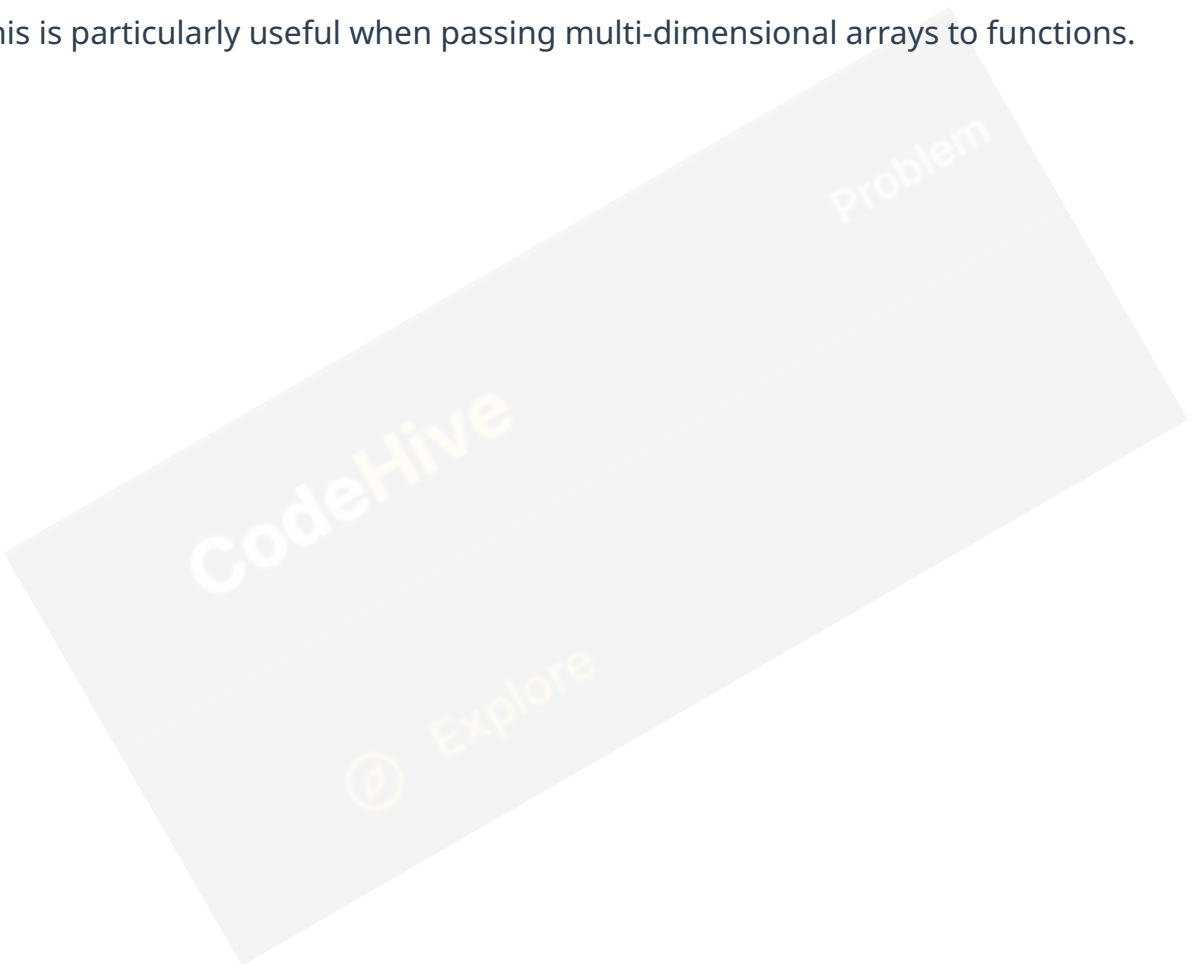
Ⓢ Explore

5. typedef with Arrays

You can even give names to specific array dimensions.

```
typedef int FiveInts[5];  
FiveInts scores = {10, 20, 30, 40, 50};
```

This is particularly useful when passing multi-dimensional arrays to functions.



6. Key Advantages ★

- **Readability:** Using `age` instead of `int` makes the purpose of the variable clear.
- **Portability:** If you need to change `int` to `long` across 100 files, you only change it once in the `typedef`.
- **Clean Code:** Simplifies complex structure and function pointer declarations.



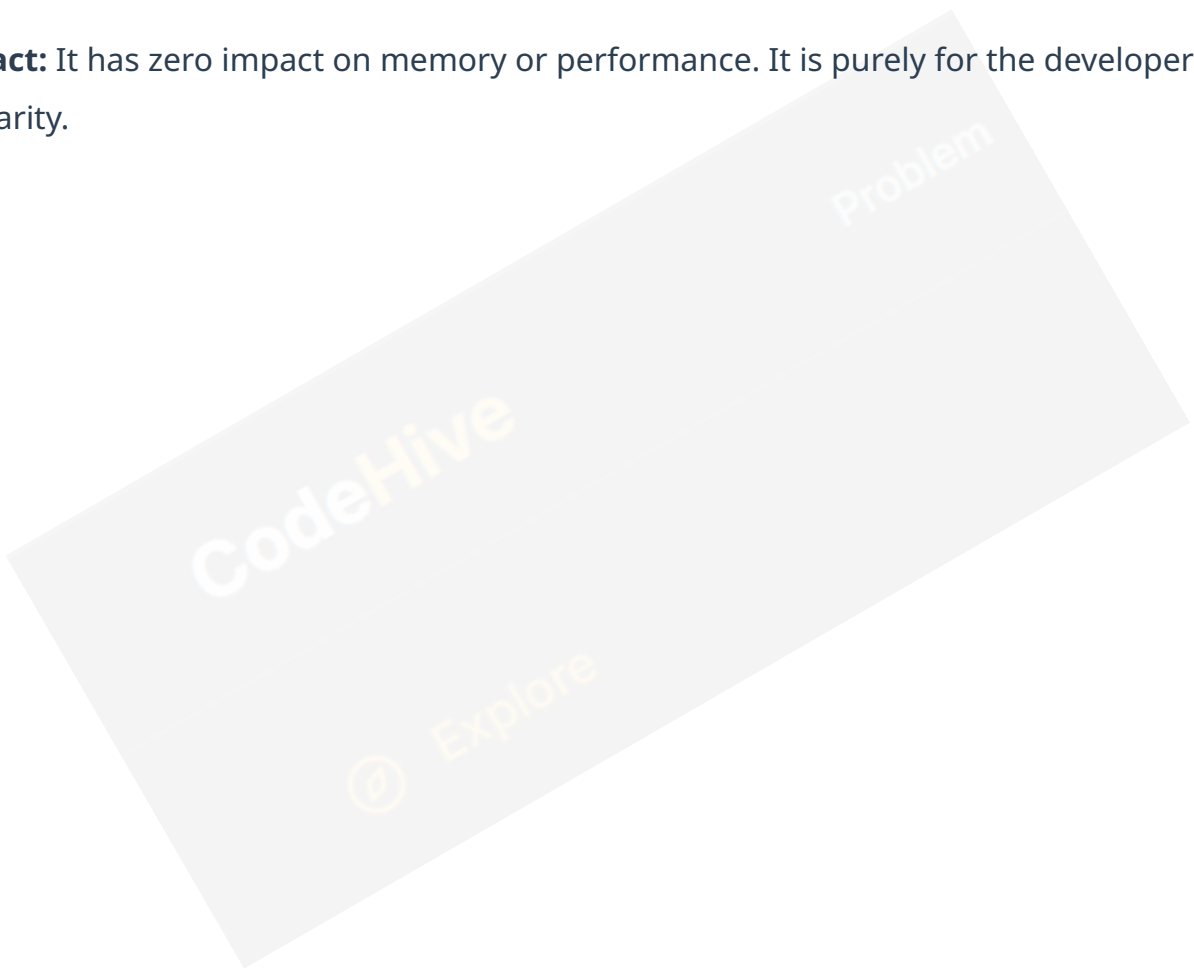
7. Myths vs. Facts

Myth: typedef creates a new data type.

Fact: It only creates an alias. The compiler treats the alias exactly like the original type.

Myth: It increases program memory.

Fact: It has zero impact on memory or performance. It is purely for the developer's clarity.



Key Points (Exam Ready)

- **typedef** is a keyword, not a preprocessor directive.
 - Widely used with **struct** to avoid the repetitive struct keyword.
 - Enhances code **maintainability**.
 - Essential for professional-grade C programming.
-

CodeHive Academy - Coding Made Simple

CodeHive

@ Explore